

Python com SQL

Artigos Alura:

1. SQL: comandos básicos
2. SQL: consultas com SELECT
3. SQL: cláusulas UPDATE e DELETE.
4. Saiba tudo sobre o que é SQL - A Linguagem padrão para trabalhar com banco de dados relacionais
5. Saber mais sobre SGBDs relacionais

Aula 01: Estruturando as tabelas

Fonte dos dados:

```
url_itens_pedidos = \
    'https://github.com/alura-cursos/SQL-python-integracao/raw/main/TABELAS/itens_pedidos.csv'
url_pedidos = \
    'https://github.com/alura-cursos/SQL-python-integracao/raw/main/TABELAS/pedidos.csv'
url_produto = \
    'https://github.com/alura-cursos/SQL-python-integracao/raw/main/TABELAS/produtos.csv'
url_vendedores = \
    'https://github.com/alura-cursos/SQL-python-integracao/raw/main/TABELAS/vendedores.csv'
```

Iremos utilizar o **SQLite** que é um banco de dados armazenados em um único arquivo, diferente dos que tem relação cliente-servidor. Oferece apenas *recursos básicos do SQL* e possui grande **portabilidade**.

A função `create_engine` serve para criar um objeto de conexão de banco de dados e o `inspect` serve para inspecionar o banco de dados e obter informações sobre suas tabelas, colunas, chaves primárias, tipos de dados, etc.

Apesar disso, outro banco poderia ser utilizado através do `create_engine`, como por exemplo:

```
engine = create_engine("postgresql://usuario:senha@localhost/banco")
```

Resumo da aula:

```
from sqlalchemy import create_engine, inspect, text
```

1. Ler um csv a partir de um link remoto e armazenar em um `pd.DataFrame`
2. Criar uma `Engine` para conectar com o banco e alocar as tabelas nele, passando a `Engine` criada com o `sqlalchemy` através do método do dataframe `df.to_sql(name, con, index=False)`
3. Criar um *inspetor* com o objeto `inspect` da `sqlalchemy` para ver as tabelas criadas com `inspetor(engine).get_tables()`

Aula 02: Primeiras Consultas

A conexão com o banco é feita por um context manager. Utilizou-se uma função auxiliar para trabalhar com DataFrame obtidos através da query:

```
def sql_df(query: str) -> pd.DataFrame:
    with engine.connect() as connection:
        consulta = connection.execute(text(query))
        dados = consulta.fetchall()
    return pd.DataFrame(dados, columns=consulta.keys())
```

Utilizamos SELECT e precisei ficar atento a precedências do *alias* como:

```
# sql_df("SELECT SUM(valor_total) from ITENS_PEDIDOS as 'Receita Total'")
#                                     erro de precedência ~~~~~
sql_df("SELECT SUM(valor_total) as 'Receita Total' from ITENS_PEDIDOS")
```

O relacionamento de duas tabelas foi feito com:

```
query = '''SELECT PRODUTOS.MARCA, COUNT (*) AS 'Pedidos'
FROM PRODUTOS, ITENS_PEDIDOS
WHERE PRODUTOS.PRODUTO_ID = ITENS_PEDIDOS.PRODUTO_ID
...
...
'''
```

O ponto e vírgula (;) serve como delimitador de instrução (statement terminator) no padrão SQL. Ele avisa ao banco de dados que aquele comando terminou e que o interpretador já pode executá-lo.

No terminal ou cliente SQL (DBeaver, pgAdmin, MySQL Workbench)

Obrigatório ao rodar múltiplos comandos: Se você enviar vários comandos no mesmo script, o ; é essencial para o banco separar uma query da outra.

Equivalente ao `df.head()` e `df.tail()` tem a mistura de cláusulas `ORDER BY id ASC` e `ORDER BY id DESC`, respectivamente.

Aula 03: Lidando com Filtros

Analisar a tabela pedidos para ver como foram as vendas dos vendedores no ano de 2021 (ano anterior). Usaram `strftime("%Y", data_compra)` para filtrar dados.

E um detalhe importante: `strftime()` não é uma função SQL universal. Ela existe no SQLite, mas outros bancos, como PostgreSQL, MySQL e SQL Server, usam funções diferentes para extrair o ano.

Com essa função, podemos extrair diferentes componentes de data e hora, por exemplo:

- Extrair o ano: '%Y';
- Extrair o mês: '%m';
- Extrair o dia: '%d'.

Exemplo:

```
## Selecionar dados do ano de 2019
```

```
query = '''
SELECT strftime('%m', data_compra) AS mes, COUNT(*) AS quantidade_vendas
FROM pedidos
WHERE strftime('%Y', data_compra) = '2019'
GROUP BY mes;
'''

vendas_19 = sql_df(query)
vendas_19
```

```
## Selecionar dados do ano de 2020
```

```
query = '''
SELECT strftime('%m', data_compra) AS mes, COUNT(*) AS quantidade_vendas
FROM pedidos
WHERE strftime('%Y', data_compra) = '2020'
GROUP BY mes;
'''

vendas_20 = sql_df(query)
vendas_20
```

Funções de Agregação

Trabalhando com funções de agregação.

AVG(), COUNT(), GROUP_CONCAT(), MAX(), MIN(), SUM() e TOTAL()

Notas: SUM() e TOTAL() desempenham a mesma função mas TOTAL() leva em consideração elementos NULL como zero. A AVG() ignora os valores nulos durante o cálculo da média. O COUNT() conta o número de linhas ou valores não nulos em uma coluna ou conjunto de valores e retorna um valor inteiro.

Aula 04: Avançando nas relações